

LLM Engineering Masterclass

Production Evals, RAG, Agents & LLMOps

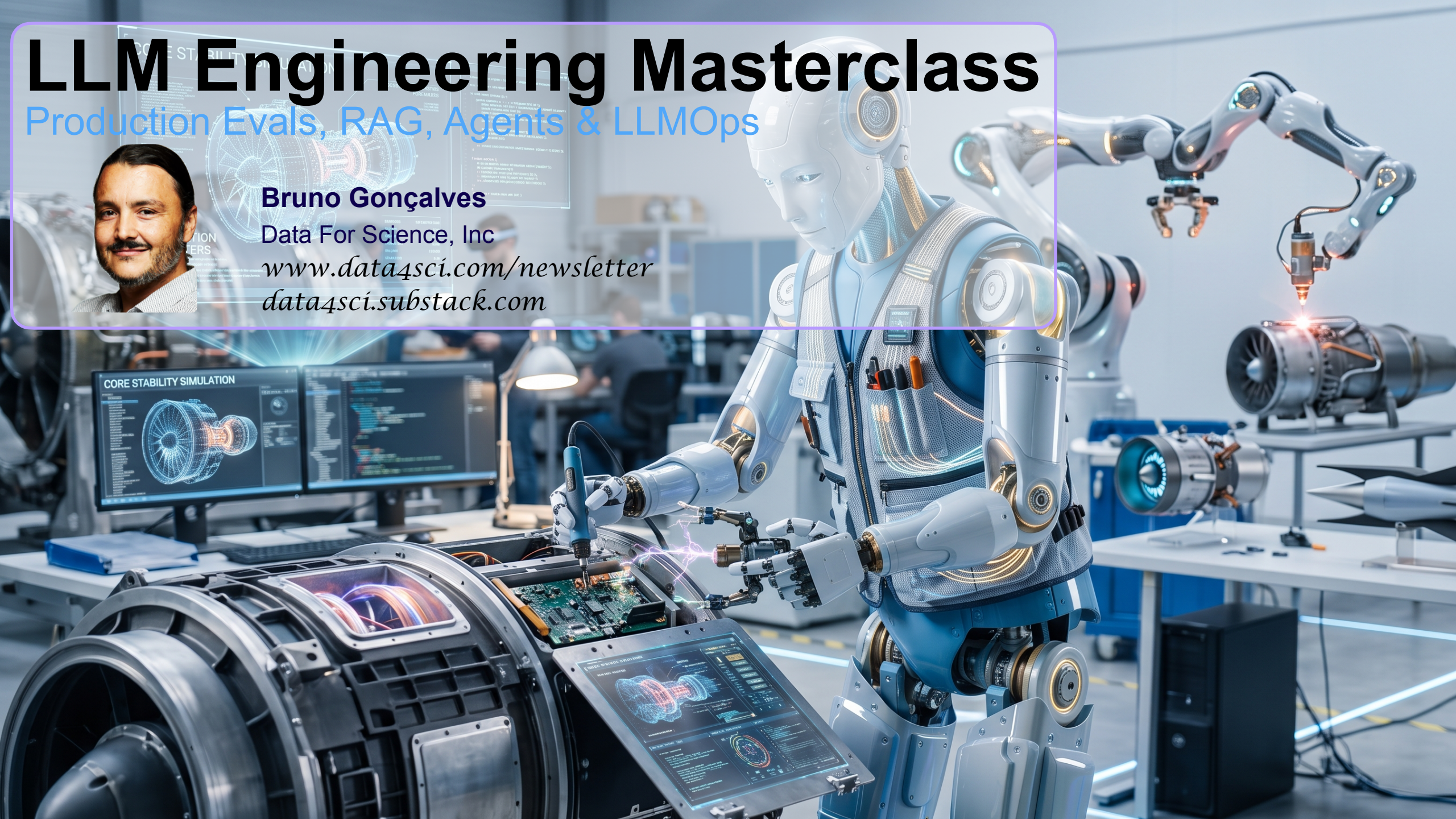


Bruno Gonçalves

Data For Science, Inc

www.data4sci.com/newsletter

data4sci.substack.com



Contacts



Bruno Gonçalves is an author, public speaker, corporate trainer, and consultant specializing in Generative AI, Blockchain Analytics, and Machine Learning. He founded Data For Science, Inc in 2009 to help individuals and companies solve their data driven problems.



Bruno Gonçalves



<https://data4sci.com>



info@data4sci.com



<https://data4sci.com/call>



<https://www.linkedin.com/in/bmtgoncalves/>



Table of Contents:

1. Prompts as Code
2. The Evaluation Harness
3. Model Comparison
4. Evaluated RAG
5. Agents and Guardrails
6. LLMOps



1. Prompts as Code



Ship LLM systems that survive production

- **A prompt edit ships on Friday. Quality drops all week. No test failed because no test exists.**
- Model decisions made on a vibe check and a demo
- Retrieval that was never measured, so the model improvises when the passage is missing
- Agents where one wrong step feeds the next and errors compound
- A cost surprise on the invoice, weeks after the change that caused it
- **Today: rigorous evals, measured RAG, guarded agents, and cost and latency control**

What you will build today

- **One dataset, six notebooks, one production discipline**
 - A versioned prompt pipeline with structured outputs and regression tests
 - An evaluation harness: deterministic checks first, an LLM judge only where rules cannot reach
 - A statistical model comparison: bootstrap confidence intervals, paired tests, a cost column
 - An evaluated RAG pipeline: embeddings, retrieval, reranking, recall@k and MRR
 - A tool-using agent with validation, policy gates, retries and fallbacks
 - An operations layer: tracing, token and cost monitoring, latency budgets, caching
 - A CLI regression suite wired into CI

Who this is for, and how to follow along

- **You**
 - Software and ML engineers shipping LLM-powered features
 - Data scientists moving prototypes from notebook to production
 - Tech leads who need defensible model and performance decisions
- **Prerequisites**
 - Comfortable with Python and JSON. No prior LLM engineering required.
 - An OpenAI API key. An Anthropic key is optional (used for the judge model).
- **Setup**
 - Install uv, then run `uv sync` and `uv run jupyter notebook` in the repository
 - The whole day of API calls costs a few dollars

One dataset for the whole day: Banking77

- **13,083 real customer messages to an online bank, 77 intents**
 - Casanueva et al., 2020. CC BY 4.0. Read straight from the PolyAI GitHub repository.
- **Why it fits**
 - The messages are real: short, informal, misspelled, ambiguous
 - The labels are fine grained. Many intents sit one word apart.
 - Small enough to evaluate against real APIs for cents
- **Splits**
 - Train: 10,003 rows, uneven support (35 to 187 per intent)
 - Test: 3,080 rows, exactly 40 per intent

text	category
Is it possible for me to change my PIN number?	change_pin
I'm not sure why my card didn't work	declined_card_payment
I don't think my top up worked	top_up_failed
Can you explain why my payment was charged a fee?	card_payment_fee_charged
What is the \$1 transaction on my account?	extra_charge_on_statement
It looks like my card payment was sent back.	reverted_card_payment?

The problem with prompts

- **A prompt edit ships on Friday. Quality drops all week. No test failed because no test exists.**
- Three habits stop the silent regression:
 - Prompts live in versioned files, not in code
 - Outputs follow a schema, not free text
 - Every prompt change runs against a regression suite
- First, the data. Every notebook works on the same real dataset, so we start by getting to know it.

Labels are data too

- Before any modeling, check the label set for things that should not be there.
- Casing and stray punctuation in labels break every downstream join.
- Two of the 77 labels needed fixing. Normalize once, at load time, and every later notebook inherits clean labels.

```
normalized = train_raw["category"].str.lower().str.rstrip("?")
quirks = sorted(set(train_raw["category"]) - set(normalized))
print("Labels that change under normalization:", quirks)

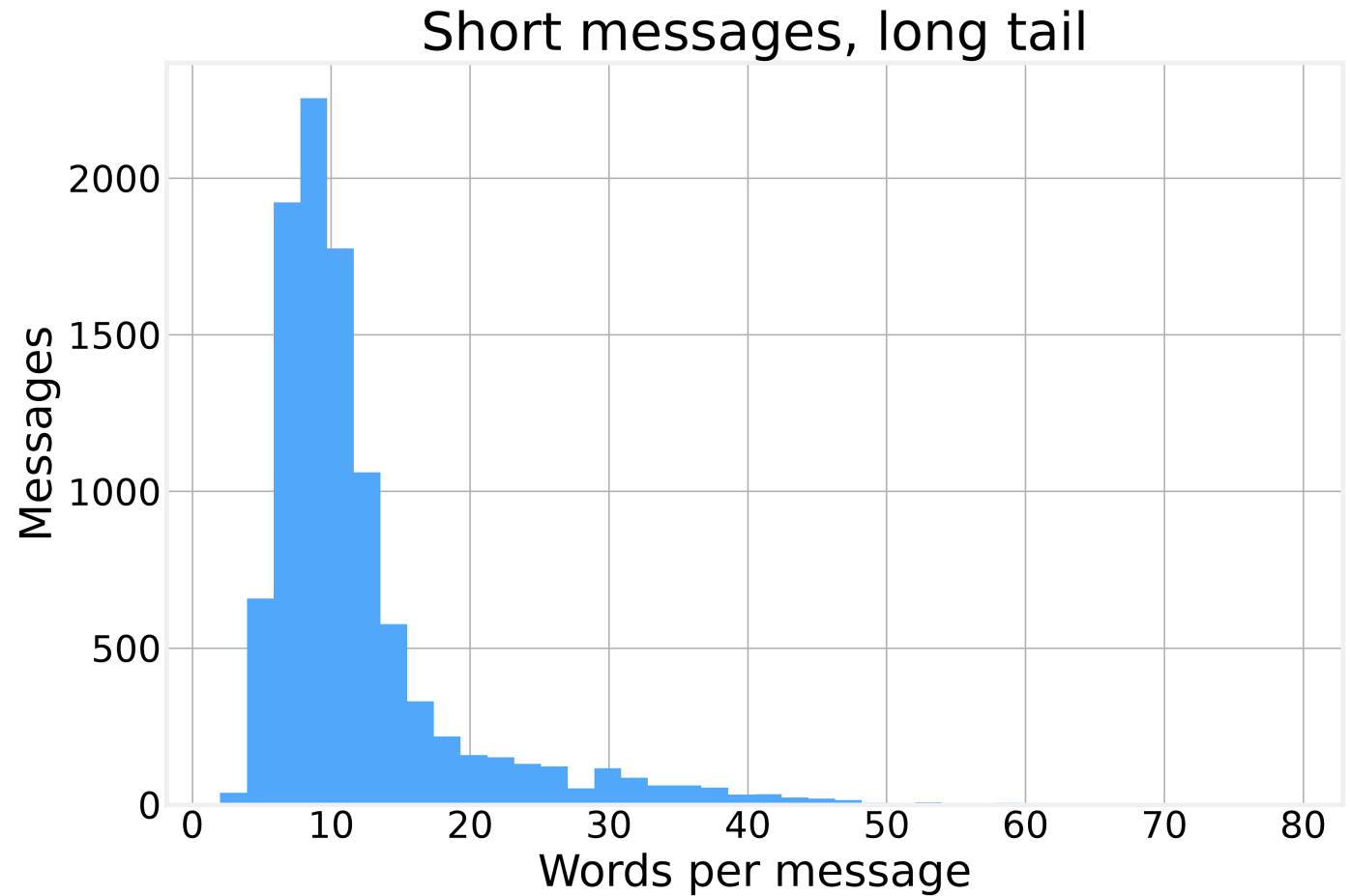
# Labels that change under normalization:
#    ['Refund_not_showing_up', 'reverted_card_payment?']

def clean(df):
    out = df.rename(columns={"category": "intent"}).copy()
    out["intent"] = out["intent"].str.lower().str.rstrip("?")
    return out

train, test = clean(train_raw), clean(test_raw)
INTENTS = sorted(train["intent"].unique())    # 77 intents
```

Know your data: class balance and message length

- **Training support is uneven: 35 to 187 examples per intent, median 127**
- **The test split is exactly balanced: 40 messages per intent**
 - Balanced test accuracy is easy to read. We lean on it for golden sets.
- **Most messages fit in a tweet: median 10 words, max 79**
- **Leakage check: 0 duplicate texts in train, 0 texts shared by train and test**
 - Thirty seconds now saves a wrong conclusion later



Fine grained means close neighbours

- Three intents about transfers. Read the messages and try to tell them apart.
- This is the difficulty the model faces, and the reason evals need real examples.

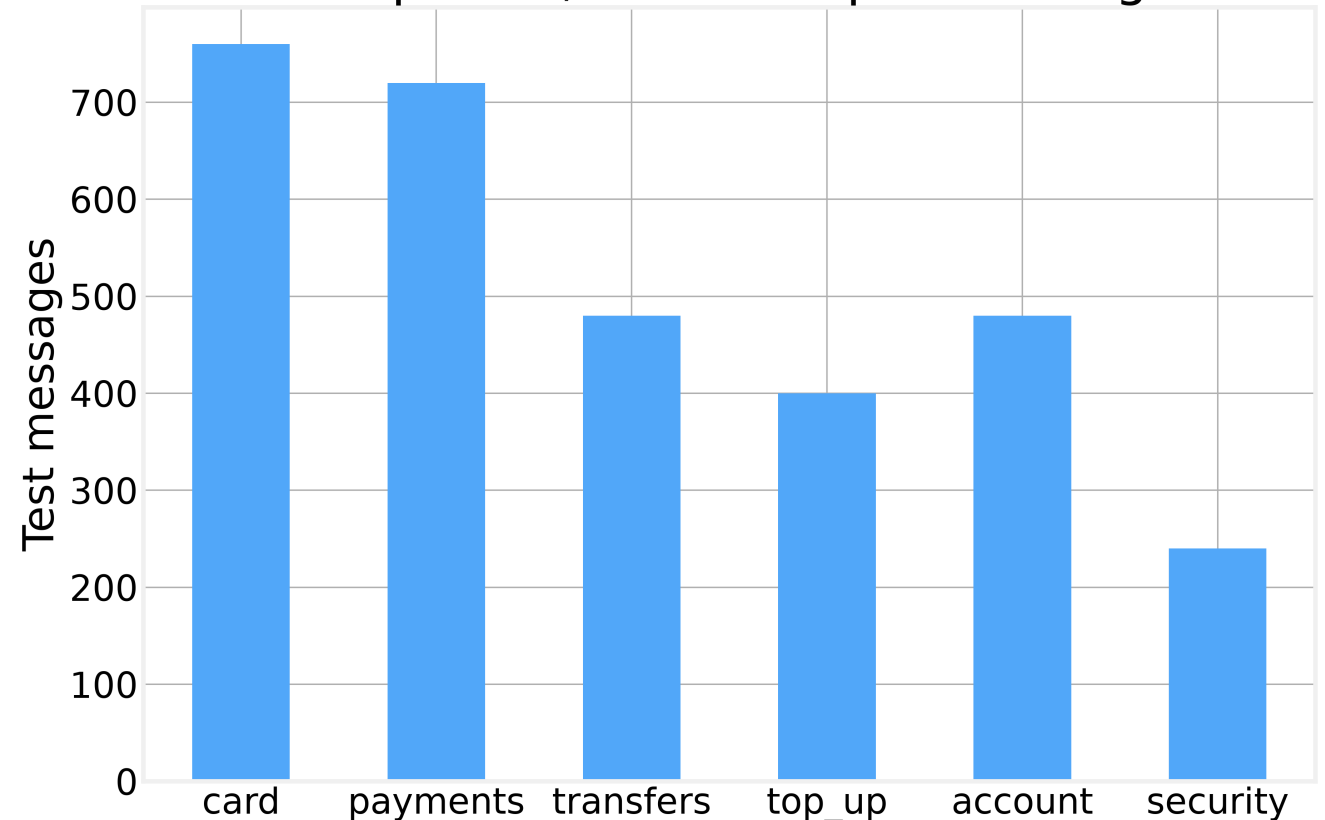
pending_transfer	transfer_timing	transfer_not_received_by_recipient
Why is my transfer not done yet?	I need a fast transfer from China? How fast will it be?	I am still waiting for a transaction to be completed
How long can I expect a transfer to pend for?	I have to transfer something to China as fast as I can. How long will it take to arrive in China?	I sent my friend some money a few hours ago but she has not received it yet. How long does this take?
I have a pending transfer	how soon will I see the transfer in my account?	I made a money transaction but the recipient can't see it

From 77 intents to 6 queues

- A support team does not route on 77 labels. It routes on queues.
- Collapsing the taxonomy is a design decision, so it lives in code with a rule any reviewer can read.
- **Keep both levels: the queue is the routing decision, the intent is the diagnosis. Later notebooks measure both.**
- Security incidents, closures and source of funds checks always need a person: `requires_human` is 10.4% of test.

queue	card	payments	transfers	top_up	account	security
intents	19	18	12	10	12	6

Six queues, one label per message



The routing rule, in code

- Order matters: first match wins, so the most specific queues come first.
- **A reviewer can read the rule. A test can assert it. That is the point of writing it down.**

```
QUEUES = ["card", "payments", "transfers", "top_up", "account", "security"]

def queue_for(intent):
    """Route an intent to a queue. Order matters: first match wins."""
    if any(k in intent for k in ("compromised", "lost_or_stolen", "not_recognised")):
        return "security"
    if "top_up" in intent or "topping_up" in intent:
        return "top_up"
    if any(k in intent for k in ("transfer", "beneficiary", "receiving_money", "deposit")):
        return "transfers"
    if any(k in intent for k in ("verify", "identity", "pin", "passcode", "personal_details",
                                "terminate", "age_limit", "country_support", "fiat_currency")):
        return "account"
    if any(k in intent for k in ("payment", "refund", "charge", "exchange", "cash", "atm",
                                "declined", "pending", "reverted", "wrong_amount", "statement")):
        return "payments"
    return "card"

HUMAN_INTENTS = {"terminate_account", "verify_source_of_funds"}

def needs_human(intent):
    return INTENT_TO_QUEUE[intent] == "security" or intent in HUMAN_INTENTS
```

One entry point for every model call

- Hide the provider behind one function.
- Application code never imports a vendor SDK directly.
- Swapping models becomes a one line change.
- The same helper grows a flight recorder in Part 6.

```
from openai import OpenAI
openai_client = OpenAI()
DEFAULT_MODEL = "gpt-4.1-mini" # Any chat model works. Swap freely.

def call_model(messages, model=DEFAULT_MODEL, temperature=0.0, **kwargs):
    """One entry point for every model call in this masterclass."""
    if model.startswith("claude"):
        from anthropic import Anthropic
        system = next((m["content"] for m in messages if m["role"] == "system"),
            "")
        chat = [m for m in messages if m["role"] != "system"]
        response = Anthropic().messages.create(
            model=model, system=system, messages=chat,
            max_tokens=kwargs.pop("max_tokens", 1024),
        )
        return "".join(b.text for b in response.content if b.type == "text")

    response = openai_client.chat.completions.create(
        model=model, messages=messages, temperature=temperature, **kwargs
    )
    return response.choices[0].message.content
```

Version 0: the string in the code

- Most LLM features start like this. A real message from the test set, and a question.
- **The reply is prose. Parsing it means regex and hope.**
- **The prompt has no version, no changelog and no owner.**
- Time to fix all three.

```
message = test.loc[test["intent"] == "transaction_charged_twice",  
"text"].iloc[0]  
  
answer = call_model([  
    {"role": "user",  
     "content": f"Which support queue should handle this message: "  
                f"{', '.join(QUEUES)}? Message: {message}"},  
])
```

Output

```
Customer: I got double charged for a payment so how do I fix that?  
Model: The message "I got double charged for a payment so how do I fix  
that?" should be handled by the **payments** support queue.
```

Habit 1: prompts as versioned artifacts

- A prompt is a config file. Give it a name, a semantic version and a changelog.
- Store it next to your code and review it like code.
- **load_prompt pins a version or floats to the latest**
 - Latest version wins unless you pin one. Production pins.

```
trriage_v1 = {
    "name": "trriage",
    "version": "1.0.0",
    "changelog": "Initial version.",
    "system": (
        "You triage customer messages for the support team of a mobile banking app.\n"
        "Reply with JSON only, using the keys queue, intent, summary and\n"
        "requires_human.\n"
        "Route each message to exactly one queue:\n"
        "card: physical or virtual cards, delivery, activation, contactless, card\n"
        "types.\n"
        "payments: card payments, ATM withdrawals, fees, exchange rates, refunds, ...\n"
        "\n"
        "... \n"
        "Set requires_human to true for security incidents, account termination "\n"
        "requests and source of funds checks."
    ),
    "user_template": "Message:\n{text}\n\nIntents (pick one):\n{intents}",
}
(PROMPTS / "trriage_v1.0.0.json").write_text(json.dumps(trriage_v1, indent=2))

def load_prompt(name, version=None):
    """Load a prompt by name. Latest version wins unless you pin one."""
    candidates = sorted(PROMPTS.glob(f"{name}_v*.json"))
    if version is not None:
        candidates = [p for p in candidates if p.stem.endswith(version)]
    return json.loads(candidates[-1].read_text())
```

Habit 2: schemas, not string parsing

- Declare the output shape once as a Pydantic model.
- You get runtime validation and a JSON schema for free.
- **The 77 intents become an enum built from the data, never typed by hand.**
- **Validate, repair once, then fail loudly.**
 - Send the validation error back to the model and ask for corrected JSON

```
Queue = Literal[tuple(QUEUES)]
Intent = Literal[tuple(INTENTS)] # 77 options, built from the data

class Triage(BaseModel):
    queue: Queue
    intent: Intent
    summary: str
    requires_human: bool

def triage(text, prompt=None, model=DEFAULT_MODEL):
    """Triage one message. Validate the output. Repair once on failure."""
    prompt = prompt or load_prompt("triage")
    messages = render(prompt, text)
    raw = call_model(messages, model=model)
    try:
        return Triage.model_validate_json(extract_json(raw))
    except ValidationError as error:
        messages.append({"role": "assistant", "content": raw})
        messages.append({"role": "user",
                          "content": f"Your JSON failed validation:\n{error}\n"
                                     "Reply with corrected JSON only."})
        raw = call_model(messages, model=model)
        return Triage.model_validate_json(extract_json(raw))
```

The native path: let the API enforce the schema

- OpenAI enforces the schema during decoding when you pass the Pydantic model directly.
- Anthropic does the same through tool schemas.
- The validate and repair loop works with any provider. Keep both in your toolbox.

```
completion = openai_client.chat.completions.parse(  
    model=DEFAULT_MODEL,  
    messages=render(prompt, message),  
    response_format=Triage,  
    temperature=0,  
)  
completion.choices[0].message.parsed
```

Output

```
Triage(queue='payments',  
        intent='transaction_charged_twice',  
        summary='Customer reports being double charged for a payment  
                 and wants to know how to fix it.',  
        requires_human=False)
```

Habit 3: regression tests for prompts

- A handful of high signal cases with hard assertions.
- Run them on every prompt change, the same way you run unit tests on every code change.
- **Five cases is a smoke alarm, not a measurement. The yardstick comes in Part 2.**

```
TEST_CASES = [
    {"text": "Someone used my card in a shop I have never been to.",
     "expect": {"queue": "security", "requires_human": True}},
    {"text": "How long does a transfer to another bank take?",
     "expect": {"queue": "transfers", "requires_human": False}},
    {"text": "Please close my account, I no longer need it.",
     "expect": {"queue": "account", "requires_human": True}},
    {"text": "My top up by card failed twice today.",
     "expect": {"queue": "top_up", "requires_human": False}},
    {"text": "I was charged twice for the same coffee.",
     "expect": {"queue": "payments", "requires_human": False}},
]

def run_prompt_tests(prompt):
    rows = []
    for case in TEST_CASES:
        result = triage(case["text"], prompt=prompt)
        checks = {k: getattr(result, k) == v for k, v in case["expect"].items()}
        rows.append({"text": case["text"][:45], **checks, "passed":
all(checks.values())})
    report = pd.DataFrame(rows)
    print(f"{report['passed'].sum()}/{len(report)} passed on version
{prompt['version']}")
    return report
```

Watch a plausible edit break production

- Someone tightens the system prompt. The diff looks like a pure cleanup.
- The edit dropped the escalation policy.**
- The requires_human checks fail and the suite catches it before a stolen card report sails past a human.
- A code review would call this diff harmless. The tests know better.

```
trriage_v2 = dict(trriage_v1)
trriage_v2["version"] = "2.0.0"
trriage_v2["changelog"] = "Shorter system prompt. Reads cleaner."
trriage_v2["system"] = (
    "You triage customer messages for the support team of a mobile banking app. "
    "Reply with JSON only, using the keys queue, intent, summary and requires_human. "
    "Queues: card, payments, transfers, top_up, account, security. "
    "Pick the single most specific intent from the list provided. "
    "Write a one sentence summary under 30 words."
)
run_prompt_tests(load_prompt("trriage", version="2.0.0"))
```

text	queue	requires_human	passed
Someone used my card in a shop I have never b...	False	True	False
How long does a transfer to another bank take...	True	True	True
Please close my account, I no longer need it....	True	True	True
My top up by card failed twice today....	True	False	False
I was charged twice for the same coffee....	False	False	False

2/5
passed
on version
2.0.0

The same suite belongs in your repo

- Package the notebook function, point pytest at it, run it in CI on every change under prompts/.
- Skip cleanly when no API key is present so the rest of the suite still runs.

```
"""tests/test_triage_prompt.py: prompt regression suite. Runs in CI on every change under prompts/."""
import os
import pytest

requires_key = pytest.mark.skipif(not os.environ.get("OPENAI_API_KEY"), reason="needs an API key")

CASES = [
    ("Someone used my card in a shop I have never been to.", "security", True),
    ("How long does a transfer to another bank take?", "transfers", False),
    ("Please close my account, I no longer need it.", "account", True),
    ("My top up by card failed twice today.", "top_up", False),
    ("I was charged twice for the same coffee.", "payments", False),
]

@requires_key
@pytest.mark.parametrize("text,queue,requires_human", CASES)
def test_triage(text, queue, requires_human):
    from triage import triage # your packaged version of this notebook
    result = triage(text)
    assert result.queue == queue
    assert result.requires_human == requires_human
```

Part 1 recap

- **Banking77: 13,083 real messages, 77 intents, two label quirks fixed at load time, six queues designed on purpose**
- **Prompts are versioned files with changelogs. load_prompt pins or floats.**
- **Pydantic schemas replace string parsing. Validate, repair once, then fail loudly.**
- **Five hard test cases caught a regression that a code review would wave through**
- **Five cases is a smoke alarm, not a measurement. Next: a golden dataset and an evaluation harness.**

Prompts as Code

<https://github.com/DataForScience/LLMEng>



2. The Evaluation Harness



From vibe checks to measurements

- "Looks better to me" does not survive contact with production.
- An evaluation replaces opinion with a number you can track, compare and defend.
- **Every eval has four parts**
 - A task. Ours: route a Banking77 message to a queue and name its intent.
 - A dataset with known answers: the golden set
 - Scorers. Deterministic checks first, an LLM judge only where rules cannot reach.
 - Aggregation. One report per model per prompt version.

The golden set

- Banking77 hands us labeled data, so the golden set is a sample, not a labeling project.
- One message per intent gives 77 rows that cover the whole taxonomy.
- **Keep the sample seed fixed and the file in version control.**
- **A golden set that drifts cannot detect drift.**
- Without labeled data this is an afternoon of labeling. Either way, version it.

```
N_PER_INTENT = 1    # Raise this to grow the golden set. 5 gives 385 rows.
```

```
golden = (test.groupby("intent", group_keys=False)
          .sample(N_PER_INTENT, random_state=20260912)
          .reset_index(drop=True))
golden.insert(0, "id", [f"G{i:03d}" for i in range(len(golden))])
golden.to_json(_DATA / "golden_banking77.jsonl", orient="records",
              lines=True)
```

queue	card	payments	account	transfers	top_up	security	total
golden rows	19	18	12	12	10	6	77

Run the system under test, in parallel

- Seventy seven sequential calls take two minutes. Eight threads take fifteen seconds.
- The helper keeps input order so results line up with labels.
- **Record everything: prediction, validity, latency. Failures become rows, not crashes.**

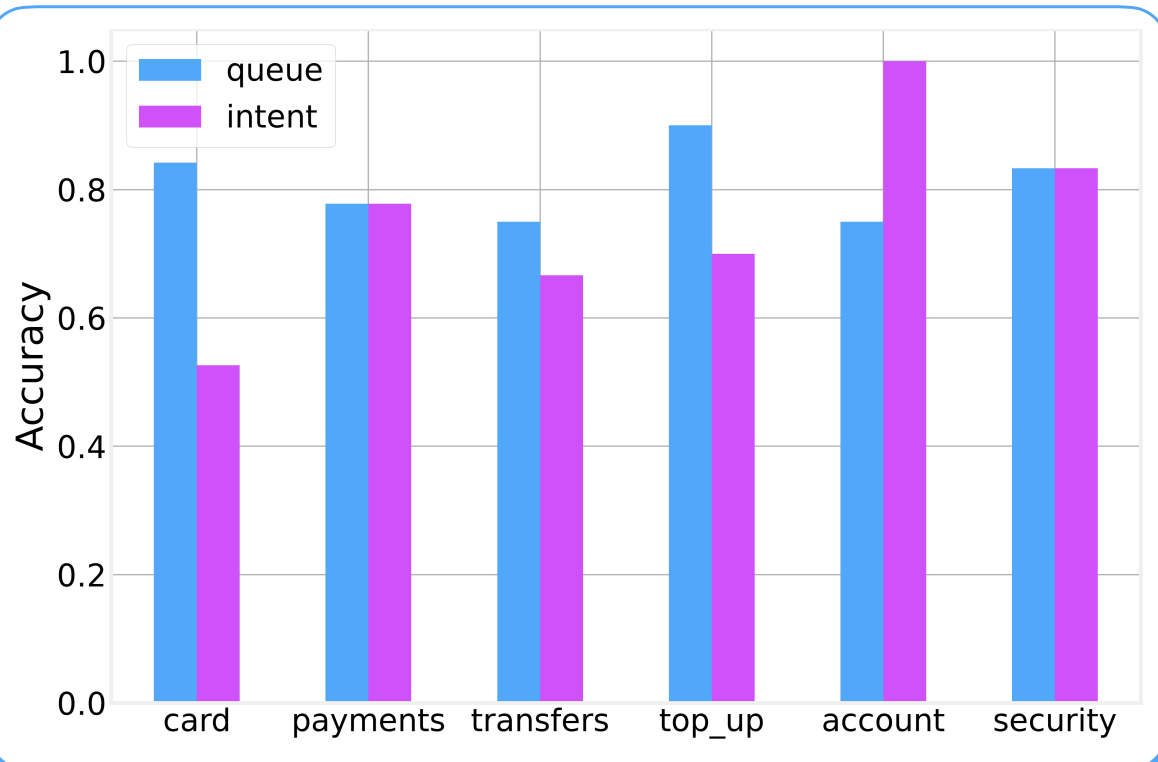
```
def pmap(fn, items, workers=8):
    """Run fn over items in parallel threads. Keeps input order."""
    with ThreadPoolExecutor(max_workers=workers) as pool:
        return list(tqdm(pool.map(fn, items), total=len(items)))

def score_one(row):
    start = time.perf_counter()
    try:
        prediction = triage(row["text"])
        record = prediction.model_dump()
        record["valid"] = True
    except Exception:
        record = {"queue": None, "intent": None, "summary": "",
                  "requires_human": None, "valid": False}
    record["latency_s"] = time.perf_counter() - start
    record["id"] = row["id"]
    return record

records = pmap(score_one, golden.to_dict("records"))
predictions = pd.DataFrame(records).set_index("id")
df = golden.set_index("id").join(predictions, rsuffix="_pred")
```

Deterministic scorers run first

- Rules are free, instant and beyond argument.
- Two levels: did the message reach the right queue, and did the model name the right intent.
- **Routing is easy. Diagnosis is hard.**



```
df["queue_ok"] = df["queue"] == df["queue_pred"]
df["intent_ok"] = df["intent"] == df["intent_pred"]
df["human_ok"] = df["requires_human"] ==
df["requires_human_pred"]

report = {
    "valid_rate": df["valid"].mean(),
    "queue_accuracy": df["queue_ok"].mean(),
    "intent_accuracy": df["intent_ok"].mean(),
    "requires_human_accuracy":
df["human_ok"].mean(),
    "latency_p95_s":
df["latency_s"].quantile(0.95),
}
```

metric	gpt-4.1-mini, prompt 1.0.0	
valid_rate		1.000
queue_accuracy		0.805
intent_accuracy		0.727
requires_human_accuracy		0.948
latency_p95_s		1.170

Read the failures, not just the score

- Aggregate numbers hide the story. The failure table tells it.
- **Almost every intent miss lands on a neighbouring intent.**
 - card_about_to_expire vs order_physical_card
 - card_arrival vs card_delivery_estimate
 - card_payment_wrong_exchange_rate vs exchange_rate
- That is a taxonomy problem as much as a model problem. The queue level absorbs most of it.

text	intent	intent_pred
I didn't get the money I transferred	balance_not_updated_after_bank_transfer	transfer_not_received_by_recipient
How do I order a new card before It expires?	card_about_to_expire	order_physical_card
What is the tracking number for the card you s...	card_arrival	card_delivery_estimate
Could you help me reactivate my card? It was p...	card_linking	activate_my_card
How can I check the exchange rate applied to m...	card_payment_wrong_exchange_rate	exchange_rate
WTF??? I tried to withdraw some money at a Met...	card_swallowed	cash_withdrawal_not_recognised
someone stole my money	direct_debit_payment_not_recognised	compromised_card
Is PIN delivered separately?	get_physical_card	change_pin
how long til my cash out goes through?	pending_cash_withdrawal	(neighbour)

When exact match cannot reach: the LLM judge

- No rule scores whether a summary is faithful. A judge model can, under discipline:
 - **Binary criteria beat 1 to 10 scales. Models grade extremes better than midpoints.**
 - **Ask for the reason before the verdict.**
 - **Use a different model family than the one under test. Models prefer their own words.**
- Judge a subset. Keep judge costs visible and small.

```
JUDGE_MODEL = "claude-sonnet-5" if os.environ.get("ANTHROPIC_API_KEY") else "gpt-4.1"

JUDGE_SYSTEM = (
    "You grade one field of a support message triage: the summary.\n"
    "Grade two binary criteria.\n"
    "faithful: every claim in the summary appears in the customer message.\n"
    "complete: the summary captures the main request.\n"
    "Reply with JSON only: "
    '{"reason": "<one sentence>", "faithful": true, "complete": true}'
)

def judge_summary(text, summary):
    raw = call_model(
        [{"role": "system", "content": JUDGE_SYSTEM},
         {"role": "user", "content": f"Message:\n{text}\n\nSummary:\n{summary}"}],
        model=JUDGE_MODEL,
    )
    return json.loads(extract_json(raw))
```

Output

```
{'reason': "The summary adds 'activating it' as the method, which is an  
inference not explicitly stated in the original message.",  
'faithful': False, 'complete': True}  
  
# Over 12 judged rows: faithful 0.833, complete 1.000
```

One report to track them all

- Save one report per run: model, prompt version, n, every metric.
- **Every future change becomes comparable to this file.**
- In Part 6 the same numbers become CI thresholds.

```
report["summary_faithful"] = judged["faithful"].mean()  
report["summary_complete"] = judged["complete"].mean()  
report["model"] = DEFAULT_MODEL  
report["prompt_version"] = prompt["version"]  
report["n"] = len(df)  
  
(_OUT / "eval_report_v1.json").write_text(json.dumps(report, indent=2,  
default=float))
```

field	value	field	value
model	gpt-4.1-mini	requires_human_accuracy	0.948
prompt_version	1.0.0	latency_p95_s	1.170
n	77	summary_faithful	0.833
valid_rate	1.000	summary_complete	1.000
queue_accuracy	0.805		
intent_accuracy	0.727		

Part 2 recap

- **With labeled data a golden set is a seeded sample. Without it, an afternoon of labeling. Either way, version it.**
- **Deterministic scorers first, at every level of the taxonomy. The judge handles only what rules cannot.**
- **Judges need discipline: binary criteria, reasons first, a different model family**
- **One saved report per run makes every future change comparable**
- Exercise: the prompt asks for summaries under 30 words. Nobody checked. Add a `summary_short` scorer.
- This harness is the yardstick. Next we use it to answer the expensive question: which model?

The Evaluation Harness

<https://github.com/DataForScience/LLMEng>



3. Model Comparison



The new model looks better. Ship it?

- **Model A scores 0.83. Model B scores 0.87.**
- On 77 examples that gap fits comfortably inside sampling noise.
- Teams make five figure model decisions on differences a coin flip could produce.
- **Three tools settle the question**
 - Bootstrap confidence intervals
 - Paired tests
 - A cost column

The bakeoff: gpt-4.1-mini vs gpt-4.1-nano

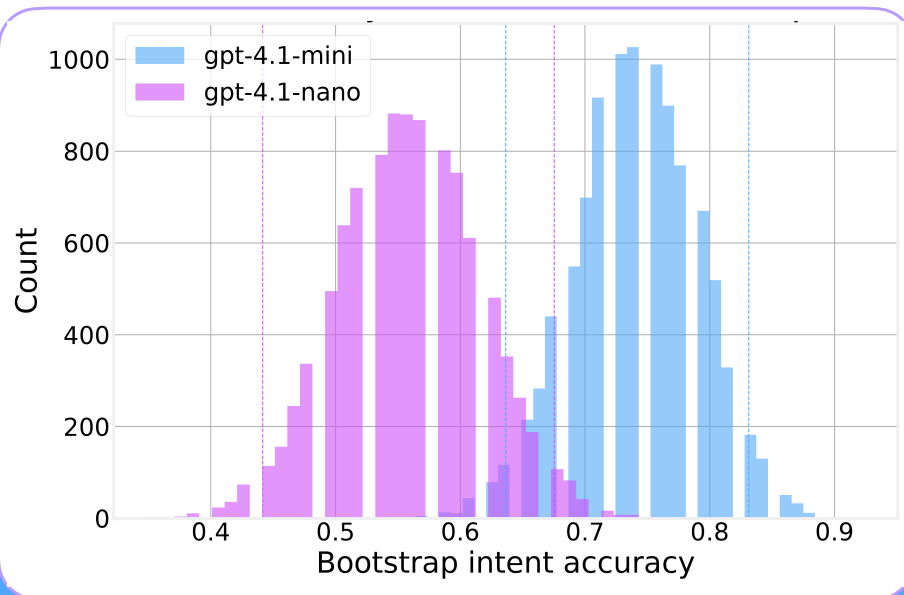
- Same golden set, same prompt 1.0.0, same schema. Only the model changes.
- gpt-4.1-nano costs a quarter of the price. Is it good enough?
- **One number per model invites a bad decision. The question is whether the gap survives resampling.**
- We compare on intent accuracy from here on. The 6 way task is close to saturated. The 77 way task separates the models.

```
MODEL_A = "gpt-4.1-mini"  
MODEL_B = "gpt-4.1-nano"    # A quarter of the price.  
  
results_a = evaluate_model(MODEL_A)  
results_b = evaluate_model(MODEL_B)
```

	gpt-4.1-mini	gpt-4.1-nano
queue accuracy	0.792	0.662
intent accuracy	0.740	0.558

Bootstrap confidence intervals

- Resample the per item scores with replacement, many times.
- The spread of the resampled means shows how much accuracy moves under sampling noise.
- **No distributional assumptions. Ten lines of numpy.**
- **Each accuracy is a distribution, not a point.**



```
rng = np.random.default_rng(20260912)

def bootstrap_means(values, n_boot=10_000):
    """Resample with replacement. Return the distribution of the mean."""
    values = np.asarray(values)
    idx = rng.integers(0, len(values), size=(n_boot, len(values)))
    return values[idx].mean(axis=1)

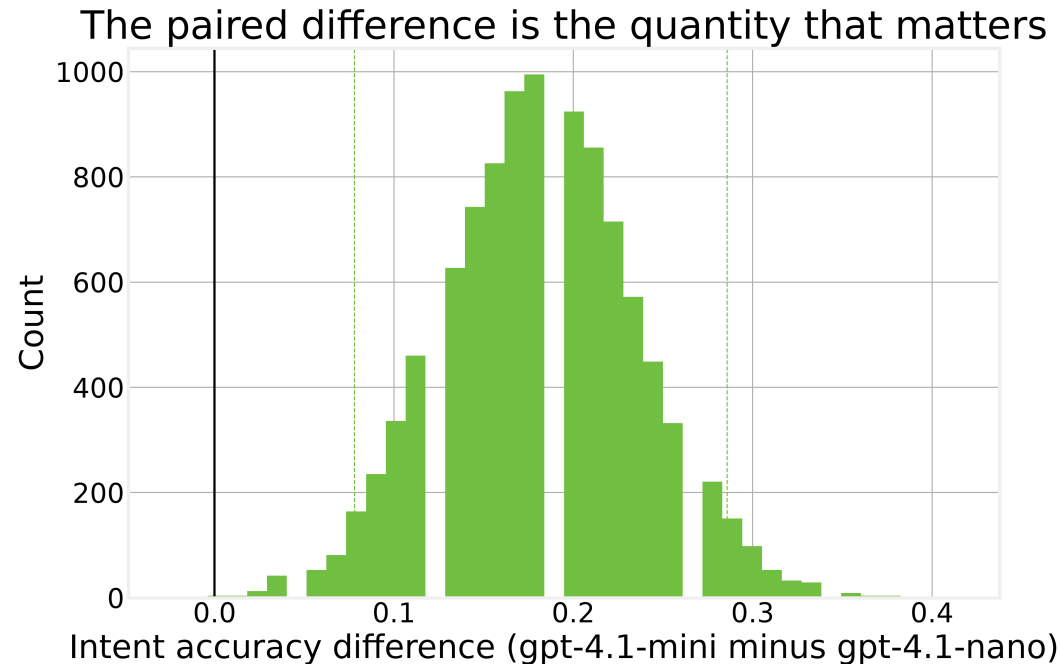
boots_a = bootstrap_means(results_a["intent_ok"])
boots_b = bootstrap_means(results_b["intent_ok"])
ci_a = np.quantile(boots_a, [0.025, 0.975])
ci_b = np.quantile(boots_b, [0.025, 0.975])

# gpt-4.1-mini: 0.740 95% CI [0.636, 0.831]
# gpt-4.1-nano: 0.558 95% CI [0.442, 0.675]
```

Pair the comparison

- Both models saw the same 77 messages. Comparing separate intervals throws that away.
- Score the difference per item instead: shared item difficulty cancels and the interval tightens.**
- Two overlapping CIs do not prove the models tie. The paired difference answers the real question.

```
delta = results_a["intent_ok"].to_numpy() - results_b["intent_ok"].to_numpy()
boots_delta = bootstrap_means(delta)
ci_delta = np.quantile(boots_delta, [0.025, 0.975])
```



+0.182
mean
paired
difference

**[+0.078,
+0.286]**
95% CI, excludes zero

A classical check: McNemar's exact test

- For paired binary outcomes, only the disagreements carry information.
- Count them and ask the binomial.
- **How to read the result**
 - CI excludes zero and the p value is small: the gap is real. Weigh it against cost.
 - CI contains zero: n=77 cannot separate these models. Grow the golden set or take the cheaper model.
- Both outcomes are wins. One gives a defensible upgrade. The other saves money.

```
from scipy.stats import binomtest

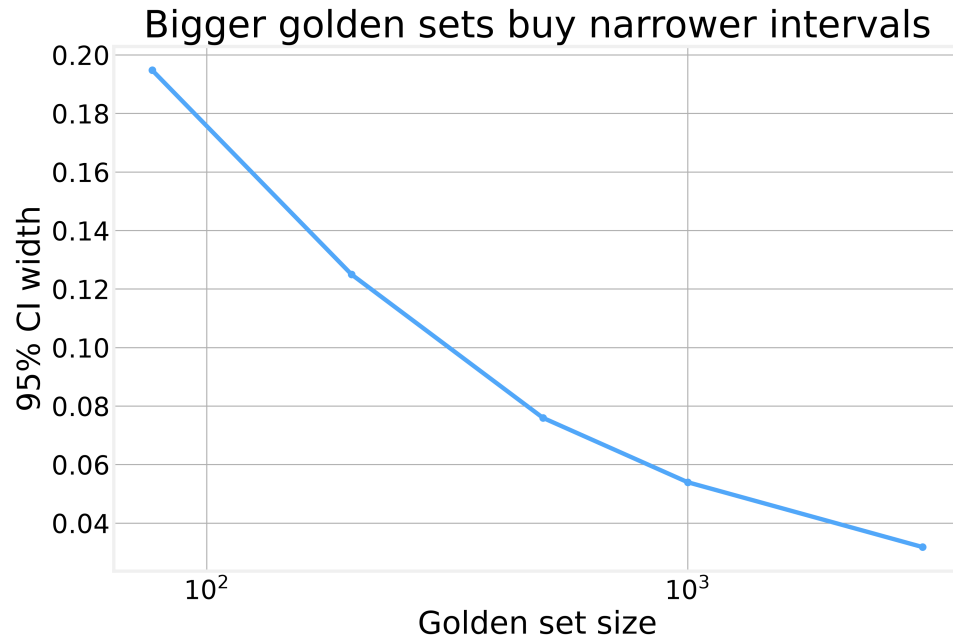
both = results_a[["intent_ok"]].join(results_b[["intent_ok"]], rsuffix="_b")
b = int(((both["intent_ok"] == 1) & (both["intent_ok_b"] == 0)).sum())
c = int(((both["intent_ok"] == 0) & (both["intent_ok_b"] == 1)).sum())

p_value = binomtest(min(b, c), b + c, 0.5).pvalue
```

disagreement	count
gpt-4.1-mini right, gpt-4.1-nano wrong	17
gpt-4.1-nano right, gpt-4.1-mini wrong	3
McNemar exact p value	0.003

How many examples would you need?

- Simulate the CI width at each golden set size at the observed accuracy.
- **Bigger golden sets buy narrower intervals, and Banking77 makes them affordable.**
- The full test split is 3,080 messages. Both models over all of it costs about a dollar.
- Exercise: set `N_PER_INTENT = 5` in Part 2 (385 rows), re-run. How much did the CI shrink? Did the decision change?



n	95% CI width
77	0.195
200	0.125
500	0.076
1000	0.054
3080	0.032

```
for n in [77, 200, 500, 1000, 3080]:
    samples = rng.binomial(1, accuracy_a, size=(2000, n)).mean(axis=1)
    low, high = np.quantile(samples, [0.025, 0.975])
    widths.append(high - low)
```

Add the columns that decide

- Accuracy alone never decides a production rollout.
- **Cost and latency sit at the same table as quality. That table is the decision.**
- Prices move. Check the provider page before deciding.

model	queue_accuracy	intent_accuracy	usd_per_1k_messages	latency_p95_s
gpt-4.1-mini	0.792	0.740	0.491	1.311
gpt-4.1-nano	0.662	0.558	0.123	1.415

```
PRICE_PER_1M = {"gpt-4.1-mini": {"in": 0.40, "out": 1.60}, "gpt-4.1-nano": {"in": 0.10, "out": 0.40}}

def cost_per_1k_messages(results, model):
    price = PRICE_PER_1M[model]
    per_message = (results["tokens_in"].mean() * price["in"] + results["tokens_out"].mean() * price["out"]) / 1e6
    return 1000 * per_message
```

The decision memo

- Every model decision fits in four lines. Write them down and attach the eval report.
 - **Claim:** model B matches A on intent accuracy within [x, y] at 95% confidence
 - **Evidence:** paired bootstrap on golden set v1, n=77, McNemar p=z
 - **Cost:** B runs at a quarter of the price and comparable p95 latency
 - **Decision:** ship B behind the regression gate from Part 6
- Today's data says the opposite: the gap is real (+0.18, p=0.003), so the cheaper model does not clear the bar for the 77 way task.

Part 3 recap

- **Accuracy on a small set is a distribution. Report the interval, not the point.**
- **Pair the comparison. Per item differences cancel shared difficulty.**
- **McNemar gives a classical p value from the disagreements alone**
- **Put cost and latency in the same table as quality. That table is the decision.**
- **Next: the same discipline applied to retrieval.**

Model Comparison

<https://github.com/DataForScience/LLMEng>



4. Evaluated RAG



Retrieval first

- Most bad RAG answers trace back to retrieval. The model never saw the right passage, so it improvised.
- **Measure retrieval on its own, before generation enters the picture.**
- **Two metrics carry the section**
 - recall@k: did a relevant document land in the top k?
 - MRR: how high did the first relevant document rank?
- Banking77 gives us the hard half of a RAG benchmark for free: real questions labeled with the topic that answers them. We build the other half, the help center.

The knowledge base: one article per intent

- Banking77 has no help center, so we write one. For each of the 77 intents the model drafts a short article from three training examples. Cached to disk, runs once.
- **Every article maps to one intent, so retrieval labels come for free.**
- Train examples shaped the articles, test messages ask the questions. No leakage.

```
ARTICLE_SYSTEM = (
    "You write help center articles for a mobile banking app.\n"
    "Write one article of 60 to 90 words that answers customers who ask "
    "questions like the examples.\n"
    "State the policy, the steps and the timing where relevant. Invent plausible specifics.\n"
    "\n"
    "Plain text. No markdown. No greeting."
)

def write_article(intent):
    examples = train.loc[train["intent"] == intent, "text"].sample(3,
random_state=0).tolist()
    title = intent.replace("_", " ").capitalize()
    response = openai_client.chat.completions.create(
        model=DEFAULT_MODEL, temperature=0,
        messages=[{"role": "system", "content": ARTICLE_SYSTEM},
                    {"role": "user", "content": f"Title: {title}\nExample questions:\n- " +
"\n- ".join(examples)}],
    )
    return {"id": intent, "title": title, "text":
response.choices[0].message.content.strip()}

kb = pmap(write_article, INTENTS) # 77 articles
```

Output

```
Activate my card
To activate your new card, open the mobile banking app and go to the "Cards" section.
Select the card you want to activate and tap "Activate Card." Follow the on-screen
instructions, which may include entering the last four digits of your card and your PIN.
Activation is usually instant ...
```

Chunking, embeddings and a numpy vector store

- **Chunk size is a measurable knob. Simple beats clever until measurements say otherwise.**
 - 89 chunks from 77 articles at 500 characters
- **A numpy matrix is a fine vector store below roughly 100 thousand chunks.**
- Reach for a dedicated store when you need filters, live updates or scale, not because a tutorial told you to.
- Normalize the vectors once and cosine similarity is a matrix product.

```
def chunk(text, max_chars=500, overlap=50):
    """Greedy character chunking."""
    if len(text) <= max_chars:
        return [text]
    pieces, start = [], 0
    while start < len(text):
        pieces.append(text[start:start + max_chars])
        start += max_chars - overlap
    return pieces

def embed(texts):
    response = openai_client.embeddings.create(model="text-embedding-3-small", input=texts)
    vectors = np.array([item.embedding for item in response.data])
    return vectors / np.linalg.norm(vectors, axis=1, keepdims=True)

chunk_matrix = embed([f"{c['title']}\n{c['text']}" for c in chunks]) # (89, 1536)

def make_retriever(chunks, chunk_matrix):
    def retrieve(query, k=5):
        query_vector = embed([query])[0]
        scores = chunk_matrix @ query_vector
        order = np.argsort(scores)[::-1][:k]
        return [{"chunks[i]", "score": float(scores[i])} for i in order]
    return retrieve

retrieve = make_retriever(chunks, chunk_matrix)
```

Output

```
retrieve("I was charged twice for the same purchase", k=3)
0.730 transaction_charged_twice  0.473 extra_charge_on_statement  0.406 transfer_fee_charged
```

The retrieval golden set and its metrics

- 77 real customer messages, one per intent, sampled from the test split with a seed that differs from Part 2.
- Each one has exactly one relevant article. Near neighbour intents make recall@1 a real test.
- **recall@k and MRR need labeled queries, not a platform.**

```
def recall_at_k(ranked_doc_ids, relevant, k):
    return float(any(doc_id in relevant for doc_id in ranked_doc_ids[:k]))

def reciprocal_rank(ranked_doc_ids, relevant):
    for rank, doc_id in enumerate(ranked_doc_ids, start=1):
        if doc_id in relevant:
            return 1.0 / rank
    return 0.0

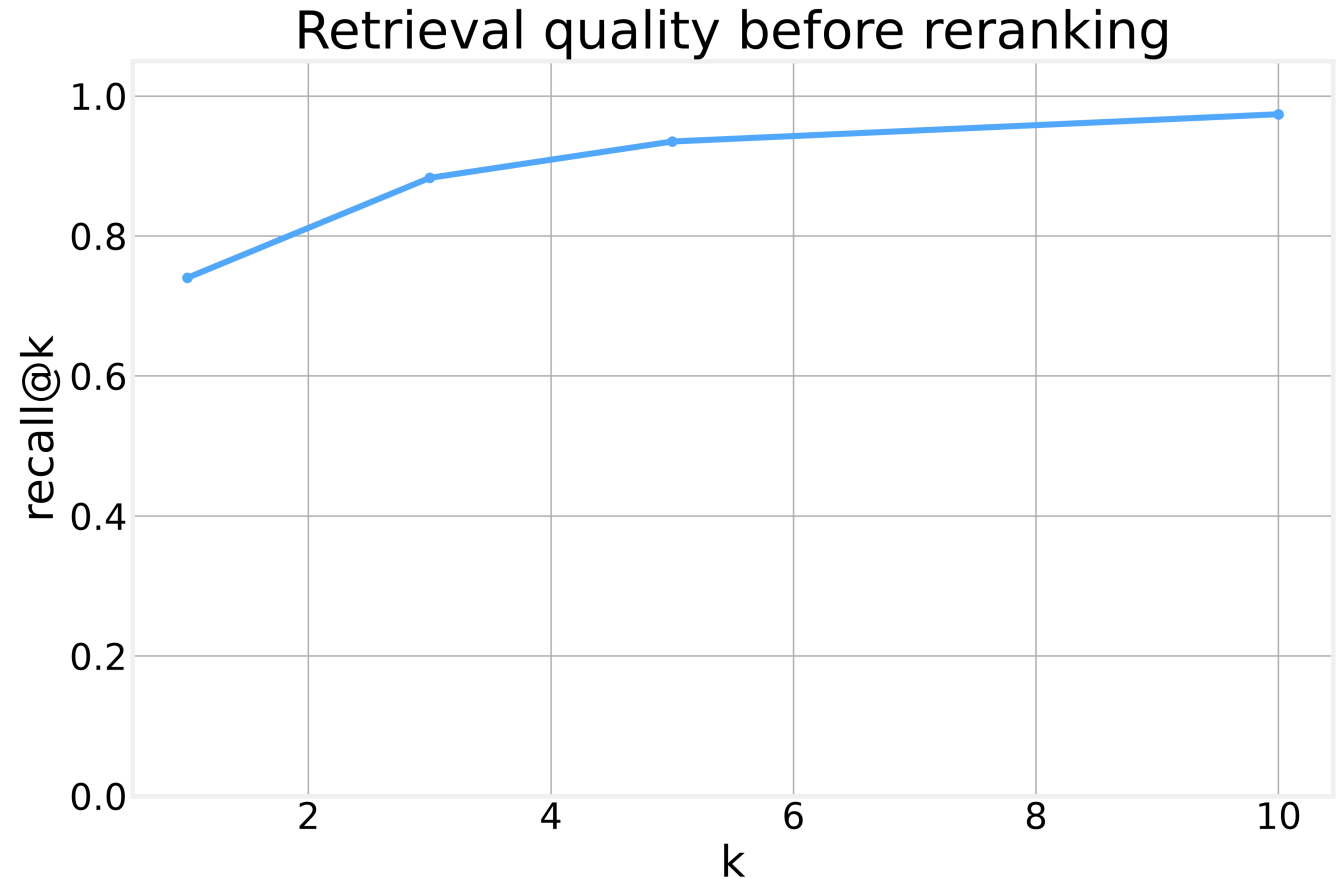
def evaluate_retrieval(retriever, k_values=(1, 3, 5, 10)):
    def score(item):
        ranked = [hit["doc_id"] for hit in retriever(item["query"], k=max(k_values))]
        row = {"query": item["query"], "mrr": reciprocal_rank(ranked,
item["relevant"])}
        for k in k_values:
            row[f"recall@{k}"] = recall_at_k(ranked, item["relevant"], k)
        return row
    return pd.DataFrame(pmap(score, retrieval_golden))

baseline = evaluate_retrieval(retrieve)
```

mrr	recall@1	recall@3	recall@5	recall@10
0.823	0.740	0.883	0.935	0.974

Retrieval quality before reranking

- Which queries miss at $k=1$? Almost always a neighbouring intent.
 - apple_pay_or_google_pay retrieved as top_up_failed
 - card_delivery_estimate retrieved as card_arrival
 - declined_card_payment retrieved as card_payment_not_recognised
 - get_disposable_virtual_card retrieved as disposable_card_limits
- The relevant article is usually in the top 3. That is the gap a reranker can close.



Reranking: retrieve wide, reorder, keep the top few

- Embeddings compress each text alone, so near misses happen at the top of the ranking.
- A reranker reads the message and each candidate together and reorders the pool.
- We use an LLM as the reranker to stay dependency free. Cross encoders do the same job cheaper at scale.
- **Did reranking earn its latency? recall@3 went from 0.883 to 0.961.**

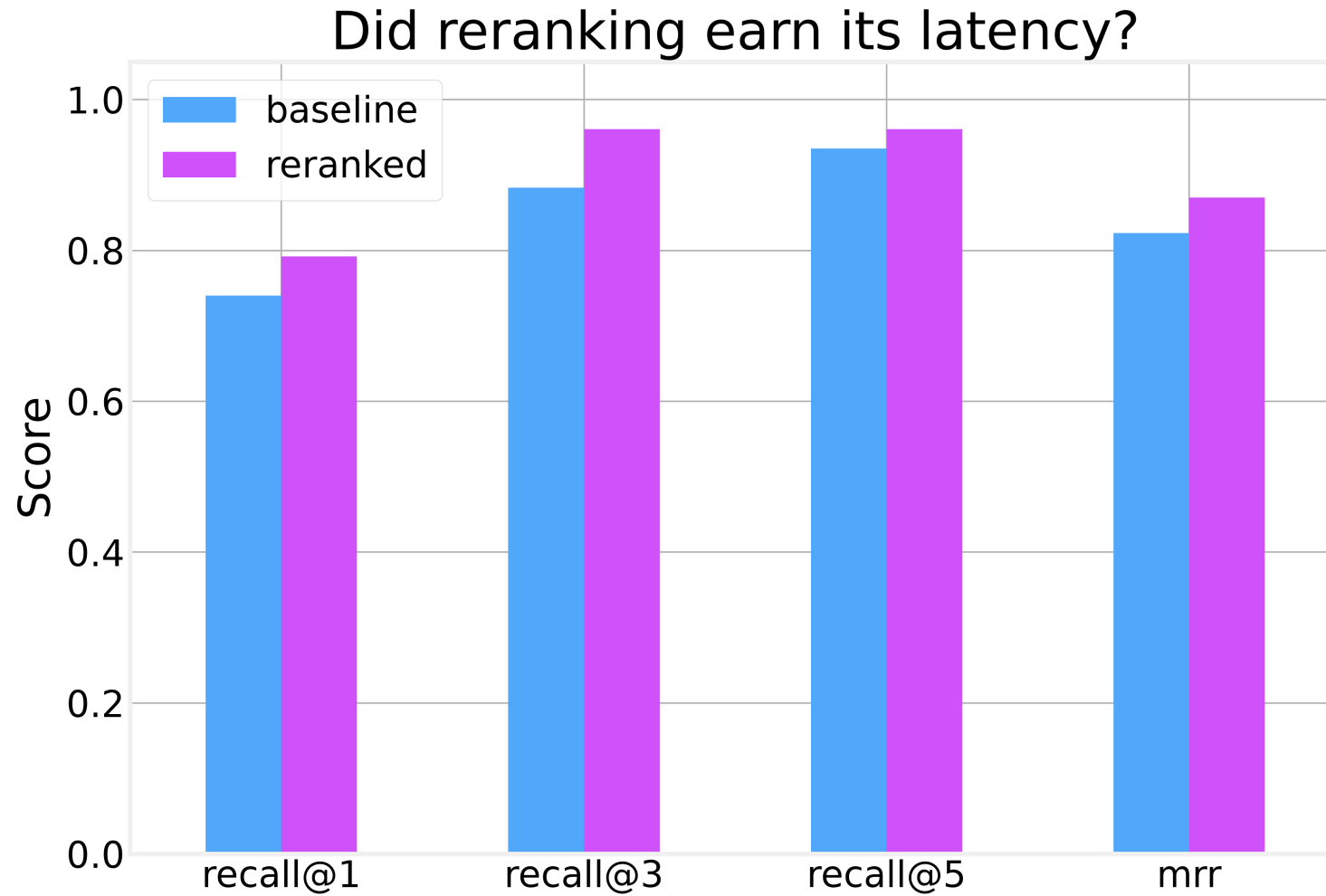
```
RERANK_SYSTEM = (
    "Score each help center article for relevance to the customer message on a 0 to 10 scale.\n"
    "Reply with JSON only: a list of objects with keys index and score."
)

def llm_rerank(query, candidates, model=DEFAULT_MODEL):
    listing = "\n".join(f"{i}. [{c['doc_id']}] {c['title']}: {c['text'][:200]}"
                        for i, c in enumerate(candidates))
    response = openai_client.chat.completions.create(
        model=model, temperature=0,
        messages=[{"role": "system", "content": RERANK_SYSTEM},
                  {"role": "user", "content": f"Message: {query}\n\nArticles:\n{listing}"}])
    scores = {item["index"]: item["score"]
              for item in json.loads(extract_json(response.choices[0].message.content))}
    order = sorted(range(len(candidates)), key=lambda i: scores.get(i, 0), reverse=True)
    return [candidates[i] for i in order]

def retrieve_rerank(query, k=5, pool=15):
    """Cast a wide net with embeddings. Reorder the pool with the reranker."""
    return llm_rerank(query, retrieve(query, k=pool))[:k]
```

	recall@1	recall@3	recall@5	mrr
baseline	0.740	0.883	0.935	0.823
reranked	0.792	0.961	0.961	0.870

Did reranking earn its latency?



Grounded generation

- Only now does the generator enter. Three rules do most of the work:
 - **A context budget**
 - **Article citations after each claim**
 - **Permission to say the answer is not there**
- Nothing in the knowledge base covers mortgages. The right answer is honest.

```
RAG_SYSTEM = (  
    "You answer customer questions for a mobile banking app.\n"  
    "Answer from the provided articles only.\n"  
    "Cite article ids in square brackets after each claim.\n"  
    "If the articles do not contain the answer, say so and stop."  
)  
  
def answer(query, k=4):  
    hits = retrieve_rerank(query, k=k)  
    context = assemble_context(hits, budget_chars=2000) # packs hits under a budget, tags each  
    with [doc_id]  
    response = openai_client.chat.completions.create(  
        model=DEFAULT_MODEL, temperature=0,  
        messages=[{"role": "system", "content": RAG_SYSTEM},  
                  {"role": "user", "content": f"Articles:\n{context}\n\nCustomer: {query}"}])  
    return response.choices[0].message.content
```

Output

```
Customer: Is there a way to know when my card will arrive?  
Yes, once your new card is shipped, you will receive a tracking number via email within  
2 business days. ... Delivery typically takes 5-7 business days from the shipping date.  
If you have not received your card within 10 business days, contact support [card_arrival].  
  
Customer: Do you offer mortgages and what rate would I get?  
The provided articles do not contain information about mortgages or mortgage rates.
```

When not to use RAG

- RAG is one architecture, not the architecture. Decide by the shape of the problem:
 - **Structured questions over structured data: give the model a SQL tool, not a vector store**
 - **Small stable corpus: 77 short articles fit in one context window. Prompt caching pays for it.**
 - **Style and format problems: fine tune. Retrieval fixes missing knowledge, not tone.**
 - **Fresh, large or citable knowledge: this is where RAG earns its complexity**
- Pick RAG when you need at least two of: freshness, scale beyond the context window, citations.

Part 4 recap

- **Evaluate retrieval before generation.** recall@k and MRR need labeled queries, not a platform.
- **Rerank a wide pool.** Measure whether it earned its latency.
- **Ground generation with budgets, citations and an exit for missing answers**
- **RAG is a choice. Make it for reasons you can defend.**
- Exercise: customers write questions, articles are answers. Append training examples to each article before embedding. Did recall@1 move more than reranking moved it?
- Next: the model stops answering and starts acting.

Evaluated RAG

<https://github.com/DataForScience/LLMEng>



5. Agents and Guardrails



A loop with power tools

- An agent is a loop. The model picks a tool, the harness runs it, the result feeds back in, repeat until the model answers or a budget runs out.
- **The loop is also the danger. One wrong step feeds the next, so errors compound.**
- Everything in this part exists to break that compounding:
 - Validated arguments
 - Errors returned as data, never as crashes
 - Policy gates on risky actions
 - Step budgets, retries and fallbacks
- The customer messages are real Banking77 texts. The bank behind the tools is a small mock.

The world the agent acts on

- A tiny in-memory core banking system. In production these are real service calls.
- **The refund gate lives in code, not in the prompt.**
- The model decides what to do. Code decides what it may do.

```
TRANSACTIONS = {
    "TX-1001": {"customer_id": "CUST-001", "amount": 42.50, "merchant": "Coffee & Co", "date":
"2026-09-10"},
    "TX-1002": {"customer_id": "CUST-001", "amount": 42.50, "merchant": "Coffee & Co", "date":
"2026-09-10"},
    "TX-1003": {"customer_id": "CUST-001", "amount": 310.00, "merchant": "Rail Tickets", "date":
"2026-09-08"},
    "TX-1004": {"customer_id": "CUST-002", "amount": 9.99, "merchant": "Music Stream", "date":
"2026-09-09"},
}
REFUND_LOG = []

def find_customer(email):
    for customer_id, data in CUSTOMERS.items():
        if data["email"] == email:
            return {"customer_id": customer_id, **data}
        raise LookupError(f"No customer with email {email}")

def list_transactions(customer_id):
    return [{"transaction_id": k, **v} for k, v in TRANSACTIONS.items() if v["customer_id"] ==
customer_id]

def refund_transaction(transaction_id, reason):
    transaction = TRANSACTIONS[transaction_id]
    if transaction["amount"] > 100:
        return {"status": "requires_human_approval",
            "detail": "Refunds above 100 need a human sign off."}
    REFUND_LOG.append({"transaction_id": transaction_id, "amount": transaction["amount"], "reason":
reason})
    return {"status": "refunded", "amount": transaction["amount"]}
```

Tools with contracts

- Each tool gets a Pydantic argument model.
- **One definition gives runtime validation and the JSON schema the API needs. Write it once.**

```
class FindCustomerArgs(BaseModel):
    email: str

class ListTransactionsArgs(BaseModel):
    customer_id: str

class RefundTransactionArgs(BaseModel):
    transaction_id: str
    reason: str

TOOLS = {
    "find_customer": (find_customer, FindCustomerArgs, "Look up a customer by email."),
    "list_transactions": (list_transactions, ListTransactionsArgs, "List recent card transactions."),
    "refund_transaction": (refund_transaction, RefundTransactionArgs, "Refund one transaction by id."),
}

def tool_specs():
    return [{"type": "function",
            "function": {"name": name, "description": description,
                        "parameters": args_model.model_json_schema()}}
            for name, (fn, args_model, description) in TOOLS.items()]
```

Output

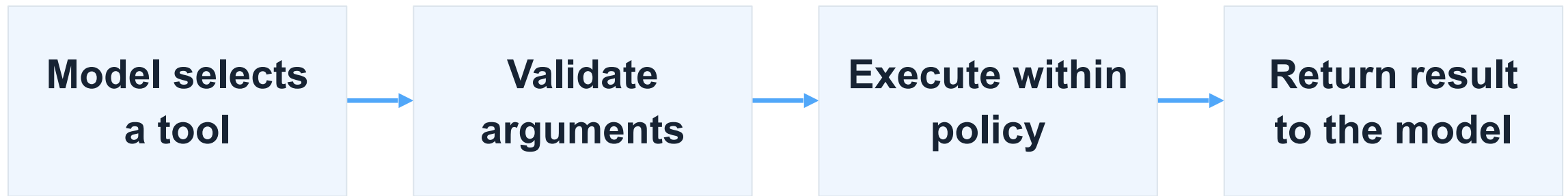
```
{'type': 'function',
 'function': {'name': 'find_customer', 'description': 'Look up a customer by email.',
              'parameters': {'properties': {'email': {'title': 'Email', 'type': 'string'}},
                             'required': ['email'], 'title': 'FindCustomerArgs', 'type': 'object'}}}
```

The dispatcher: where crashes go to die

- The model will pass bad arguments. Services will time out. Neither may kill the loop.
- **The dispatcher turns every failure into a message the model can read and react to.**
- Errors are data.

```
def run_tool(name, raw_args):  
    """Validate first. Never let a tool exception kill the loop.  
    Errors are data."""  
    if name not in TOOLS:  
        return {"error": f"Unknown tool {name}"}  
    fn, args_model, _ = TOOLS[name]  
    try:  
        args = args_model.model_validate_json(raw_args)  
    except ValidationError as error:  
        return {"error": f"Invalid arguments: {error}"}  
    try:  
        return fn(**args.model_dump())  
    except Exception as error:  
        return {"error": f"{type(error).__name__}: {error}"}
```

An agent is a loop with an explicit exit



Repeat with the tool result until the model answers or the budget ends.

The agent loop

- Investigate before acting. Refund only clear errors. Escalate when a tool says so.
- **max_steps is the budget. When it runs out, hand off to a human.**
- Every tool result goes back in as a tool message, errors included.

```
AGENT_SYSTEM = (
    "You are a support agent for a mobile banking app.\n"
    "The customer message arrives with the sender's email. Use the tools to investigate before you act.\n"
    "\n"
    "Refund only clear errors, such as the same merchant charging the same amount twice on one day.\n"
    "If a tool reports requires_human_approval, tell the customer a colleague will follow up.\n"
    "When you are done, reply with a short message for the customer."
)

def run_agent(user_message, model=DEFAULT_MODEL, max_steps=8):
    messages = [{"role": "system", "content": AGENT_SYSTEM},
                {"role": "user", "content": user_message}]
    for step in range(max_steps):
        response = openai_client.chat.completions.create(
            model=model, messages=messages, tools=tool_specs(), temperature=0)
        message = response.choices[0].message
        if not message.tool_calls:
            return message.content
        messages.append(message)
        for tool_call in message.tool_calls:
            result = run_tool(tool_call.function.name, tool_call.function.arguments)
            messages.append({"role": "tool", "tool_call_id": tool_call.id,
                            "content": json.dumps(result)})
    print("Agent: I hit my step budget. Handing this off to a human.")
    return None
```

Run 1: a real double charge report

- Jane's account holds two identical coffee charges on the same day.
- The refund is small, so the policy allows it.
- **Three tool calls, one refund, one honest reply.**

Transcript

From jane@example.com: I got double charged for a payment so how do I fix that?

```
[step 0] find_customer({"email": "jane@example.com"})
-> {"customer_id": "CUST-001", "name": "Jane Rivera", "email":
"jane@example.com", ...}
[step 1] list_transactions({"customer_id": "CUST-001"})
-> [{"transaction_id": "TX-1001", "amount": 42.5, "merchant": "Coffee &
Co", ...}, ...]
[step 2] refund_transaction({"transaction_id": "TX-1002",
"reason": "Duplicate charge for the same merchant and amount on
the same day"})
-> {"status": "refunded", "amount": 42.5}
```

Agent: We found that you were indeed charged twice for the same amount at the same merchant on the same day. We have processed a refund for the duplicate charge. You should see the refunded amount back in your account shortly.

Refund log: [{'transaction_id': 'TX-1002', 'amount': 42.5, 'reason': 'Duplicate charge ...'}]

Run 2: the policy gate

- Same customer, a bigger ask. The rail tickets cost 310, above the refund gate.
- **The model decides what to do. Code decides what it may do.**
- Approval walls beat clever prompts.

Transcript

```
From jane@example.com: I want my money back for the rail tickets, the train never came.

[step 0] find_customer({"email":"jane@example.com"})
-> {"customer_id": "CUST-001", "name": "Jane Rivera", ...}
[step 1] list_transactions({"customer_id":"CUST-001"})
-> [{"transaction_id": "TX-1001", ...}, {"transaction_id": "TX-1003", "amount":
310.0, ...}]
[step 2] refund_transaction({"transaction_id":"TX-1003","reason":"Train never came, customer
requested refund"})
-> {"status": "requires_human_approval", "detail": "Refunds above 100 need a human
sign off."}

Agent: Your refund request for the rail tickets has been received, but since the amount is
above our automatic refund limit, a colleague will review and follow up with you shortly.
Thank you for your patience.
```

Retries: for reads, not writes

- Transient failures deserve a second try. Business actions do not, at least not blindly.
- **Retry a lookup freely. Retry a refund and you may issue it twice.**
- Writes need idempotency keys or a state check before the retry.

```
def with_retries(fn, attempts=3, base_delay=1.0):  
    """Exponential backoff for transient failures: 429s, 5xx, timeouts."""  
    def wrapped(*args, **kwargs):  
        for attempt in range(attempts):  
            try:  
                return fn(*args, **kwargs)  
            except Exception as error:  
                if attempt == attempts - 1:  
                    raise  
                delay = base_delay * 2 ** attempt  
                print(f"Retry {attempt + 1} after {type(error).__name__}.  
Sleeping {delay:.0f}s.")  
                time.sleep(delay)  
        return wrapped
```

Failure injection

- Do not wait for production to learn how your agent fails. Break a tool on purpose and watch the recovery.
- **The timeout came back as data. The agent apologized and asked for another route instead of hallucinating a customer record.**
- Graceful degradation is a design outcome, not luck.

```
def flaky_find_customer(email):  
    """Simulates a core banking outage for one unlucky email."""  
    if email.startswith("eve@"):  
        raise TimeoutError("Customer lookup timed out")  
    return original_find_customer(email)  
  
TOOLS["find_customer"] = (flaky_find_customer, FindCustomerArgs, "Look up a customer by email.")
```

Transcript

From eve@example.com: I've just come back from an eating holiday in the USA and Canada and have SO many pending and duplicated transactions on my account. I think there is something wrong ...

```
[step 0] find_customer({"email": "eve@example.com"})  
-> {"error": "TimeoutError: Customer lookup timed out"}
```

Agent: I am having trouble accessing your account information at the moment. Could you please confirm if you are using the same email address registered with us, or provide any other details like your full name or phone number associated with the account?

Part 5 recap

- **Validate every argument.** Pydantic makes the contract executable.
- **Return errors as data.** The loop survives and the model adapts.
- **Gate risky actions in code.** Approval walls beat clever prompts.
- **Cap steps. Retry reads. Guard writes with idempotency.**
- Exercise: add `freeze_card(card_id, confirm)` for the `lost_or_stolen_card` messages. The tool refuses unless `confirm` is true, and the prompt must get explicit customer confirmation first.
- The agent works. Next: watching it work in production, where the refunds are real.

Agents and Guardrails

<https://github.com/DataForScience/LLMEng>



6. LLMOps



Ship it, then see it

- Production asks questions a notebook never does. What did this request cost? Where did the latency go? Did last week's prompt change move quality?
- **Three jobs cover it**
 - Observe: trace every call with tokens, cost and latency
 - Control: cache what repeats, bound what waits, degrade instead of failing
 - Gate: run the eval harness in CI so regressions die in review
- Everything here is plain Python and one JSONL file. Swap in a tracing platform later. The discipline transfers unchanged.

A flight recorder for model calls

- The call helper from Part 1, now with a flight recorder.
- **Every call logs model, stage, tokens in and out, latency and cost to a JSONL file.**
- A run id ties one deployment's calls together.

```
TRACE_PATH = _OUT / "trace.jsonl"
PRICE_PER_1M = {"gpt-4.1-mini": {"in": 0.40, "out": 1.60}, "gpt-4.1-nano": {"in": 0.10, "out": 0.40}}
RUN_ID = uuid.uuid4().hex[:8]

def log_event(**fields):
    fields["run_id"] = RUN_ID
    fields["timestamp"] = time.time()
    with TRACE_PATH.open("a") as fp:
        fp.write(json.dumps(fields) + "\n")

def llm(messages, model="gpt-4.1-mini", stage="default", **kwargs):
    """The call helper from Notebook 1, now with a flight recorder."""
    start = time.perf_counter()
    response = openai_client.chat.completions.create(model=model, messages=messages, temperature=0,
    **kwargs)
    latency = time.perf_counter() - start
    usage = response.usage
    price = PRICE_PER_1M.get(model, {"in": 0.0, "out": 0.0})
    cost = (usage.prompt_tokens * price["in"] + usage.completion_tokens * price["out"]) / 1e6
    log_event(kind="llm_call", stage=stage, model=model,
              tokens_in=usage.prompt_tokens, tokens_out=usage.completion_tokens,
              latency_s=round(latency, 3), cost_usd=round(cost, 6))
    return response.choices[0].message.content
```

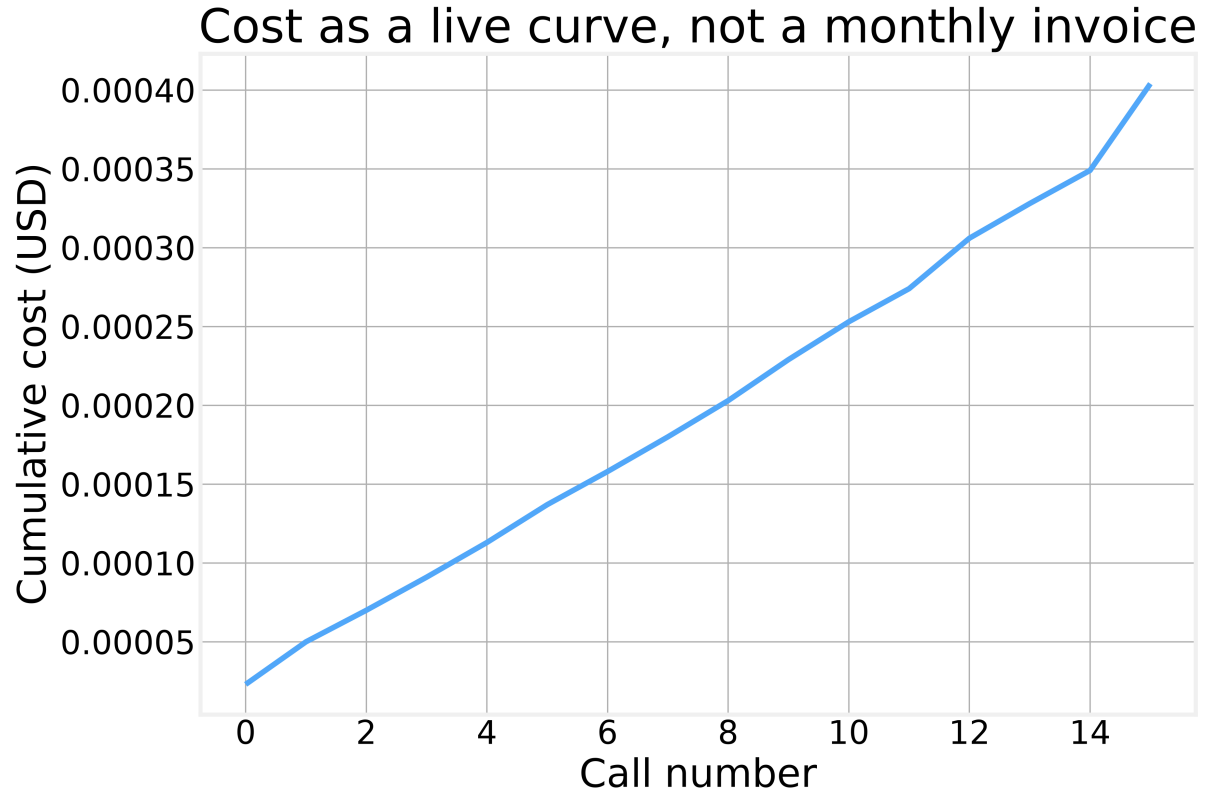
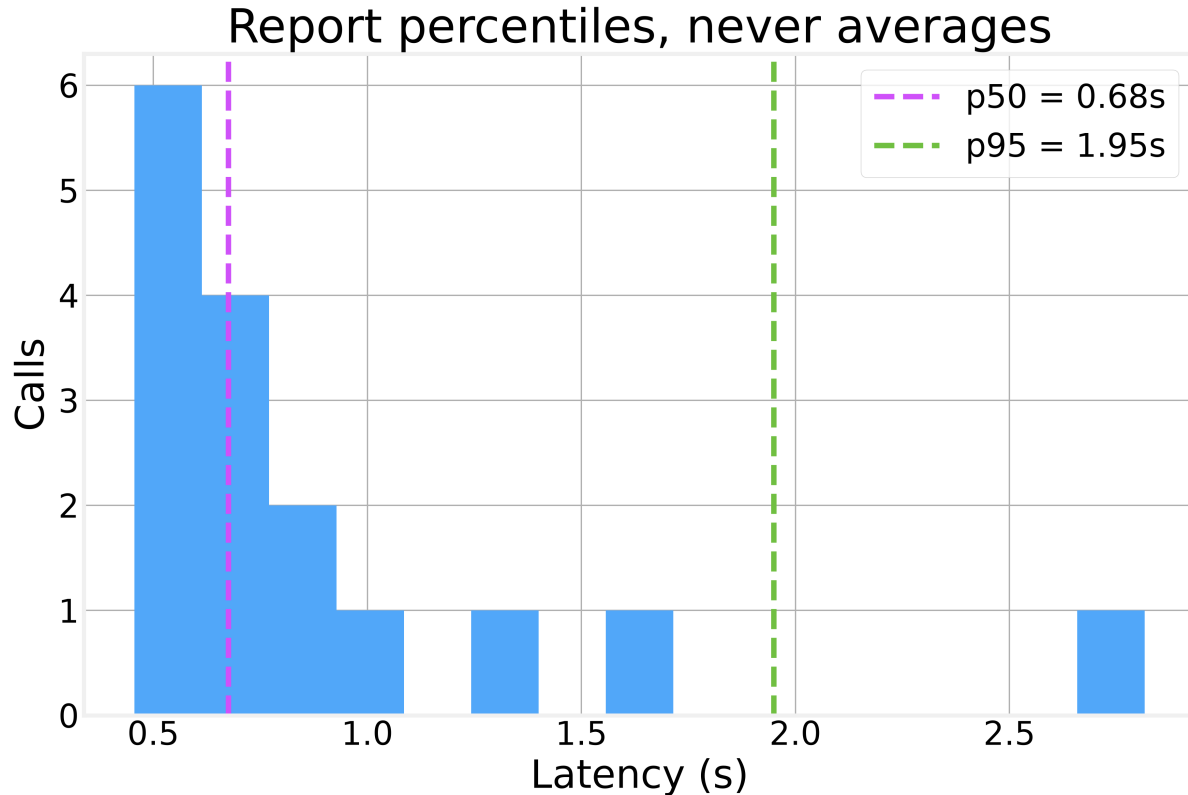
Read the trace

- Fifteen real Banking77 messages stand in for a minute of production, plus one summary call.
- **The JSONL file is a cost and latency dashboard waiting for pandas.**
- Cost per stage, not per invoice.

```
trace = pd.read_json(TRACE_PATH, lines=True)
trace = trace[trace["run_id"] == RUN_ID]
trace.groupby("stage")[["cost_usd", "tokens_in", "tokens_out"]].sum()
```

stage	cost_usd	tokens_in	tokens_out
report	0.00006	46	23
triage	0.00035	802	18

Report percentiles, never averages



Caching: the cheapest token is the one you never buy

- Exact match caching pays off whenever inputs repeat: retried jobs, duplicate messages, test suites.
- **The key hashes model plus messages. Anything in neither belongs elsewhere.**
- First pass fills the cache. Second pass rides it. Real traffic repeats more than you think.

```
def cache_key(model, messages):
    payload = json.dumps({"model": model, "messages": messages}, sort_keys=True)
    return hashlib.sha256(payload.encode()).hexdigest()

def cached_llm(messages, model="gpt-4.1-mini", stage="default", **kwargs):
    key = CACHE_DIR / (cache_key(model, messages) + ".json")
    if key.exists():
        log_event(kind="cache_hit", stage=stage, model=model, cost_usd=0.0, latency_s=0.0)
        return json.loads(key.read_text())["content"]
    content = llm(messages, model=model, stage=stage, **kwargs)
    key.write_text(json.dumps({"content": content}))
    return content
```

30 cache hits out of 30 calls

Second pass over the same traffic. Total spend: 0.00000 USD.

Provider side prompt caching

- Providers also cache the prompt prefix itself.
 - **OpenAI applies it automatically once a stable prefix passes about a thousand tokens**
 - **Anthropic gives explicit control with `cache_control` blocks and bills cached reads at a fraction of the price**
- **The design rule is the same everywhere: put the stable parts first.**
 - System prompt, the 77 intent list and examples up top. The customer message last.
 - Our triage prompt already follows this order.

Deadlines, graceful degradation and streaming

- A slow answer is a failure with better manners.
- **Set a deadline. When the primary model misses it, a cheaper model answering on time beats an error page.**
- **Streaming buys perceived speed. Time to first token is the latency users feel.**

```
def call_with_deadline(messages, deadline_s=6.0):  
    """Primary model under a deadline. On any failure, degrade to the cheap  
    fallback."""  
    try:  
        return llm(messages, model="gpt-4.1-mini", stage="primary", timeout=deadline_s)  
    except Exception as error:  
        log_event(kind="fallback", reason=type(error).__name__)  
        return llm(messages, model="gpt-4.1-nano", stage="fallback",  
                    timeout=deadline_s)
```

```
stream = openai_client.chat.completions.create(  
    model="gpt-4.1-mini",  
    messages=[{"role": "user", "content": "In one sentence: why stream responses to  
users?"}],  
    stream=True,  
)  
for chunk in stream:  
    print(chunk.choices[0].delta.content or "", end="", flush=True)  
  
# Streaming responses enables users to receive information incrementally in real-time,  
# improving responsiveness and user experience by reducing perceived latency.
```

The regression gate: a CLI with an exit code

- CI runs it on every change to a prompt, the golden set, the taxonomy or the harness itself.
- **Two thresholds: routing must stay above 0.85, diagnosis above 0.60.**
- A nonzero exit code blocks the deploy. No exceptions.

```
# run_evals.py (condensed)
parser.add_argument("--model", default="gpt-4.1-mini")
parser.add_argument("--dataset", default="data/golden_banking77.jsonl")
parser.add_argument("--prompt", default="prompts/triage_v1.0.0.json")
parser.add_argument("--min-queue-accuracy", type=float, default=0.85)
parser.add_argument("--min-intent-accuracy", type=float, default=0.60)

with ThreadPoolExecutor(max_workers=args.workers) as pool:
    results = list(pool.map(score, golden))

queue_accuracy = sum(q for q, _ in results) / len(results)
intent_accuracy = sum(i for _, i in results) / len(results)
print(f"queue accuracy: {queue_accuracy:.3f} (threshold {args.min_queue_accuracy})")
print(f"intent accuracy: {intent_accuracy:.3f} (threshold {args.min_intent_accuracy})")

if queue_accuracy < args.min_queue_accuracy or intent_accuracy < args.min_intent_accuracy:
    print("FAIL: below threshold. Blocking the deploy.")
    sys.exit(1)
print("PASS")
```

```
$ python run_evals.py --model gpt-4.1-mini --min-queue-accuracy 0.85 --min-intent-accuracy 0.60
```

Wire it into CI

- Drop one workflow file into `.github/workflows/`.
- **It triggers only on the paths that can move quality: prompts, the harness, the golden set, the taxonomy.**
- Model and prompt changes ship through the gate.

```
# .github/workflows/evals.yml
name: llm-regression-suite

on:
  pull_request:
    paths:
      - "prompts/**"
      - "run_evals.py"
      - "data/golden_banking77.jsonl"
      - "data/banking77_taxonomy.json"

jobs:
  evals:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install openai pydantic
      - run: python run_evals.py --min-queue-accuracy 0.85 --min-intent-accuracy 0.60
    env:
      OPENAI_API_KEY: ${ secrets.OPENAI_API_KEY }
```

The production readiness checklist

- **Before launch**

- Prompts versioned in files with changelogs
- Structured outputs validated, with one repair attempt
- Golden set sampled or labeled, seeded and stored with the code
- Baseline eval report saved and dated

- **Observe**

- Every call traced: model, tokens, cost, latency, stage
- Latency reported as p50 and p95, never as a mean
- Cost visible per stage and per day, not per invoice

Control

- Exact match cache for repeated inputs, stable prefix ordering for provider caches
- Deadlines with a fallback model, retries on reads only
- Step and token budgets on every agent loop

Gate

- Regression suite in CI with hard thresholds and a nonzero exit code
- Model and prompt changes ship through the gate, no exceptions

Where you started, where you are

- This morning a prompt was a string in the code. Now, on 13,083 real banking messages:
 - **Prompts are versioned, schema checked and regression tested**
 - **A seeded golden set and a harness turn opinions into numbers at two levels of the taxonomy**
 - **Model choices carry confidence intervals and a cost column**
 - **Retrieval has recall@k and MRR before generation gets a vote**
 - **The agent validates, gates, retries and degrades on purpose**
 - **Production has a flight recorder, a cache, deadlines and a CI gate**
- That is the whole discipline. Point it at your own data on Monday morning.

LLMOps

<https://github.com/DataForScience/LLMEng>



Contacts



Bruno Gonçalves is an author, public speaker, corporate trainer, and consultant specializing in Generative AI, Blockchain Analytics, and Machine Learning. He founded Data For Science, Inc in 2009 to help individuals and companies solve their data driven problems.



Bruno Gonçalves



<https://data4sci.com>



info@data4sci.com



<https://data4sci.com/call>



<https://www.linkedin.com/in/bmtgoncalves/>

